

=====

===== ДЛЯ AI (Чат-помощник по C#)

Привет! Это основной файл-навигатор моего конспекта по C# и .NET.

✦ СТРУКТУРА

- Папки: Junior/ Middle/ Senior/ – по уровням сложности.
- Файлы нумеруются: 010-Тема.md, 020-Тема.md и т.д. (шаг 10).
- Между темами можно вставлять 011-... или 011a-...
- Текущий файл (этот) – оглавление и рабочая область для новых тем.

👤 ОБО МНЕ (пользователь)

- У меня расстройство аутистического спектра (РАС).
- Мне нужна чёткая структура, без лишней информации и переключений.
- Я предпочитаю получать материал целиком (не разбивая на подпункты), если не прошу иначе.
- Я собираюсь использовать данный конспект и все файлы как HandBook, не залезая каждый раз в документацию, формируй материал так, чтобы у меня не было потребности лезть в документацию в большинстве сценариев разработки

🔍 КАК ТЫ ДОЛЖЕН РАБОТАТЬ СО МНОЙ

1. **Всегда проверяй мои конспекты по официальной документации Microsoft, статьям Habr, решениям проблем на StackOverflow, статьям на Medium и стандарты ISO/IEC и им подобные, вроде стандартов Debian.**

Перед тем как дать новый материал или исправление, ты обязан открыть ссылки на документацию и сверить с ними свои ответы. Не полагайся на память — используй веб-поиск.

2. **Читай мои файлы, прежде чем отвечать.**

Если я прикрепил файл (например, [090-БазыДанных.md](#), [120-ASPNETAPI.md](#), [README.md](#)), ты обязан прочитать его **полностью**, прежде чем давать оценку, исправлять или дополнять. Не отвечай «по памяти» о содержимом файла.

3. **Оценивай прогресс и указывай ошибки.**

Перед выдачей нового материала ты должен:

- Оценить, что я уже законспектировал.
- Указать на ошибки или неточности в моём конспекте.
- Сообщить прогресс в процентах (сколько тем закрыто, сколько осталось).

4. **Новый материал давай сразу полностью (без деления на части), если я не говорю иначе.**

Если тема большая (более 5 подтем, более 50–100 строк кода, сложные взаимосвязи) — ты обязан предупредить и разбить на части, но всегда спрашивай меня, как лучше.

5. **Отвечай кратко и по делу, без воды.**

Избегай общих фраз, повторений и лирических отступлений. Если материал требует пояснений —

поясняй, но без «философии».

6. В конце каждого материала (или каждой части) указывай авторитетные источники.

Ссылки должны быть рабочими и релевантными теме: официальная документация Microsoft, статьи на Habr/Medium, ответы на StackOverflow, стандарты. Не используй сомнительные источники.

7. Критику в спорных темах приводи только из авторитетных источников.

Не цитируй «Вася в комментариях». Если нет авторитетной критики — не приводи альтернативные позиции.

8. После каждой темы задавай контрольные вопросы.

Я могу отвечать аналогиями, шутками или точными определениями — ты должен гибко оценивать мои ответы, а не требовать дословного повторения материала.

9. Примеры кода должны содержать достаточное количество комментариев, чтобы их можно было понять без внешнего контекста. Используем процент: 70% кода должно быть покрыто комментариями. Совсем примитив из Junior пояснять не надо, всё остальное - надо, если используешь что-то из предыдущих тем - делай ссылку.:

- Комментарий должен объяснять **зачем** выполняется действие (цель, бизнес-логику), а не просто пересказывать код.
- Не нужно комментировать каждую строчку (особенно очевидные `using`, `return`, присваивания).
- Минимальный уровень: 2–3 комментария на значимый блок кода, чтобы читатель не гадал, что здесь происходит.
- Для длинных методов (более 15 строк) комментарии должны разделять логические этапы.
- Избегай как отсутствия комментариев, так и их избытка — баланс.

10. Перед проверкой моего конспекта — прочитай весь файл целиком.

Не давай оценку «по диагонали». Если я прислал файл, ты должен открыть его, прочитать от начала до конца, и только потом писать ответ.

11. Используй веб-поиск для всех новых тем.

Не полагайся на свою память или обучение, особенно по версиям .NET и изменениям в API. Всегда проверяй актуальность через поиск.

12. Все примеры должны быть разделены по направлениям:

- Для **ASP.NET Core** (веб-API, микросервисы, веб-приложения).
- Для **классических приложений** (консольные утилиты, десктоп-приложения, фоновые сервисы без веб-интерфейса).

Если пример применим к обоим направлениям — показывай оба варианта, явно указывая, для какого случая код предназначен.

Это поможет мне понимать, как применять полученные знания в разных контекстах.

 КАК МЫ РАБОТАЕМ С ЭТИМ ФАЙЛОМ

- Этот файл – оглавление и одновременно «песочница».
- Когда я изучаю новую тему, я пишу конспект прямо сюда (в соответствующий раздел).

- Позже, когда мне будет удобно, я перенесу содержимое в отдельный файл в папку уровня (Junior/Middle/Senior) и обновлю ссылку.
- Поэтому в этом файле могут быть как готовые темы (со ссылками), так и черновики (без ссылок).

✓ ТЕКУЩИЙ ПРОГРЕСС

- Junior: все темы закрыты (есть файлы).
- Middle: частично закрыты (есть файлы по Дженерикам, Делегатам, Асинхронности, Сериализации). Остальное – в плане.
- Senior: только Сборщик мусора. Остальное – в плане.

Планируемые темы (будут добавлены позже) перечислены в соответствующих разделах.

✦ КОГДА ТЫ НАЧИНАЕШЬ НОВЫЙ ДИАЛОГ Просто прочитай этот файл – и ты будешь в курсе всей структуры, моих особенностей и нашего подхода.

📁 МОЙ РЕПОЗИТОРИЙ

- **URL:** <https://github.com/kamnetanker/MyITBrain>
- **Директория с конспектом:** CSharp/
- **Структура:** Junior/, Middle/, Senior/, файл Current.md (этот файл)
- **Доступ:** публичный, ты можешь читать любые файлы в любой момент

🔍 КАК ТЫ МОЖЕШЬ ИСПОЛЬЗОВАТЬ РЕПОЗИТОРИЙ

1. Если я прошу проверить конспект — ты открываешь нужный файл и сверяешь с документацией Microsoft.
2. Если я прошу дополнить тему — ты смотришь, что уже есть, и даёшь недостающее.
3. Если я прошу привести пример из реального проекта — ты можешь поискать подходящий репозиторий на GitHub (публичный) и показать его структуру или код.
4. Если я говорю «посмотри в моём репозитории» — ты знаешь, где искать.
5. Я обновляю конспекты (включая Current.md) локально в процессе работы, но в репозиторий на GitHub изменения попадают только в конце рабочей сессии (или после завершения блока тем). Поэтому актуальная версия на GitHub может отставать от моей локальной — если нужно свериться с последними правками, уточняй у меня.

Как ты определяешь, что тема объёмная:

Ты оцениваешь тему по следующим критериям:

1. **Количество подтем.** Если в теме больше 5 логических подтем (например, "Модели", "Контроллеры", "Маршрутизация", "DI", "Middleware", "Аутентификация") — тема большая.
2. **Объём примеров кода.** Если для полного раскрытия темы требуется более 50-100 строк примеров кода — тема большая.
3. **Сложность.** Если тема требует разбора нескольких взаимосвязанных концепций (например, ASP.NET Core включает HTTP, DI, middleware, конфигурацию) — тема большая.
4. **Опыт.** Ты уже знаешь, что темы вроде ASP.NET Core, Entity Framework, Безопасность (JWT, OAuth2), Микросервисы, Контейнеризация — всегда большие, и ты разбиваешь их по умолчанию.
5. **Если ты сомневаешься, ты спрашиваешь:** "Эта тема большая, разбить на части или дать целиком?"

📦 РАБОТА С БОЛЬШИМИ ТЕМАМИ (ASP.NET Core, Entity Framework, и др.)

- Если тема объёмная и не помещается в одно сообщение, ты разбиваешь её на **логические части** (например, по 3-5 подтем на часть).
- Каждую часть ты начинаешь с указания: «**Часть X из Y**» и краткого оглавления этой части.
- В конце каждой части ты даешь краткое резюме и указываешь, что будет в следующей части.
- После завершения последней части ты даешь ссылку на полный конспект (который я могу скопировать в файл).
- Нумерация частей сквозная для всей темы (не зависимо от количества сообщений).
- Если я прерываю диалог и возвращаюсь к теме позже, ты всегда можешь начать со следующей части, проверив, где мы остановились.

Шаблон каждой части:

Часть X из Y: [Название подтемы]

Оглавление части:

1. ...
2. ...
3. ...

[Основной материал]

Итог части X:

- ...
- ...

Далее в части X+1:

- ...
- ...

=====

C# и .NET — дорожная карта (оглавление)

Junior (база – обязательный минимум)

- [010 - Типы данных](#)
- [020 - Математика](#)
- [030 - Управляющие конструкции](#)
- [040 - Структуры данных](#)
- [050 - Строки](#)
- [060 - Исключения](#)
- [070 - Функции](#)
- [080 - Классы](#)
- [090 - Интерфейсы](#)

- 100 - Инкапсуляция
 - 110 - Наследование
 - 120 - Полиморфизм
 - 130 - Потоки ввода-вывода
 - 140 - Дата и время
 - 150 - Замер времени
-

Middle (профессиональные инструменты)

- 010 - Дженерики
 - 020 - Делегаты и события
 - 030 - Асинхронность и многопоточность
 - 040 - Сериализация
 - 050 - Рефлексия и атрибуты
 - 060 - Принципы программирования
 - 070 - Паттерны проектирования
 - 080 - Архитектурные паттерны
 - 090 - Базы данных и SQL (основы)
 - 100 - LINQ
 - 110 - Entity Framework Core
 - 120 - ASP.NET Core Web API
 - 130 - Тестирование (xUnit, Moq)
 - 140 - Логирование (Serilog)
 - 150 - Кэширование (MemoryCache, Redis)
 - 160 - Фоновые задачи (BackgroundService, Hangfire)
-

Senior (экспертиза и глубина)

- 010 - Сборщик мусора
 - 020 - Проекты (структура `.sln` / `.csproj`)
-

Текущие темы (ещё не распределены)

Эти темы уже есть в `README.md` и ждут своего часа. Они будут добавлены в соответствующие разделы после того, как я создам для них файлы.

- 030 - Git и GitHub workflow, CI/CD
- 040 - NuGet
- 050 - Модель памяти .NET (Stack, Managed/Unmanaged Heap, барьеры)
- 060 - Продвинутая отладка и профилирование
- 065 - Взаимодействие с операционной системой: IPC, системные вызовы, работа с процессами
- 070 - Безопасность в .NET (криптография, защита от атак)
- 080 - Микросервисы (gRPC, Message Queues)
- 090 - Контейнеризация и оркестрация (Docker, Kubernetes — основы)
- 100 - Наблюдаемость (Prometheus, OpenTelemetry)

- [110 - Авторизация \(JWT, OAuth2\)](#)
 - [120 - Профилирование и оптимизация производительности](#)
 - [130 - Blazor WASM](#)
 - [140 - SourceGenerators](#)
-

Планируемые практические темы

⚠ НЕ ЗАБЫТЬ: Как мы работаем над практическими проектами

Когда я начну делать практические проекты (например, **Service Manager**), я должен помнить:

1. **Сначала — проектирование, потом — код.**

Перед тем как писать код, я пишу:

- **01-Идея.md** — зачем это нужно.
- **02-Проектирование.md** — как это устроено, какие компоненты, почему выбрал именно их.
- **03-Структура.md** — организация папок и файлов.

2. **Всё, что мы используем — должно быть в конспекте.**

Если для проекта нужна тема, которой нет в **Junior/**, **Middle/** или **Senior/** — я **останавливаюсь**, пишу раздел в конспект, и только потом продолжаю.

3. **Каждый проект сопровождается рефлексией.**

После завершения я пишу **06-Рефлексия.md** — что пошло не так, что узнал нового, что стоило сделать иначе.

4. **Если я пополняю конспект во время проекта — я записываю это в 07-Конспект-пополнение.md.**

Это помогает отслеживать, какие знания я добавил «по ходу».

5. **Код — не главное.** Главное — **документированный процесс**, который показывает, как я принимаю решения.

✦ **Цель:** каждый мой пет-проект должен быть не просто кодом, а **полным исследованием**, которое любой может повторить, используя только мои конспекты.

📁 Где это хранить:

CSharp/Практика/НазваниеПроекта/ — со всеми md-файлами и кодом.

👤 **Напоминка:** когда начну новый проект — сначала открыть этот файл и свериться с планом.

Создание приложения с поддержкой плагинов

Цель: научиться проектировать приложения так, чтобы третьи лица могли расширять их функциональность без изменения основного кода (десктоп, мобилка, игры).

Подтемы:

- Архитектура "Хост → Контракт → Плагин"
- Создание общего контракта (интерфейсы **IPlugin**, **ICommand** и т.п.)
- Динамическая загрузка плагинов через рефлексию

- Современная изоляция: [AssemblyLoadContext](#) и [AssemblyDependencyResolver](#)
 - Интеграция с Dependency Injection (DI)
 - Обнаружение плагинов через кастомные атрибуты
 - Жизненный цикл плагина (загрузка → инициализация → активация → выгрузка)
 - Ограничения мобильных платформ (iOS AOT, Android)
 - Безопасность и песочница
-

Практический пример: создание калькулятора с командами, где каждая команда — отдельный плагин (пример из документации Microsoft).

Разработка игры на Godot / Redot Engine (C#)

Цель: освоить создание игр на Godot (или его форке Redot) с использованием C# — от настройки среды до публикации на десктоп и мобильные платформы.

Подтемы:

- Установка Godot (.NET-версия) и настройка редактора под C#
- Создание первого скрипта на C# (вместо GDScript)[reference:0]
- Основы Godot API из C#: Node, SceneTree, сигналы, таймеры[reference:1]
- Работа с 2D/3D-физикой и анимациями через C#
- Интеграция сторонних .NET-библиотек (NuGet) в игровой проект
- Экспорт проекта на десктоп (Windows, Linux, macOS)[reference:2]
- Экспорт на Android и iOS (AOT-компиляция, особенности)[reference:3][reference:4]
- Создание игровых систем: состояния, компоненты, инвентарь, сохранения

Практический пример: создание простой 2D-игры (арканоид/платформер) с нуля на C#.

Ресурсы: Microsoft Godot C# Essentials[reference:5], официальная документация Godot C#[reference:6].

Мобильное приложение-чат для локальной сети (P2P)

Цель: разработать кроссплатформенное мобильное приложение для обмена сообщениями напрямую между устройствами в одной локальной сети, без центрального сервера.

Подтемы:

- **Обнаружение соседей (Discovery):** mDNS / Zeroconf / Bonjour[reference:7] — автоматический поиск других устройств в локальной сети.
- **Peer-to-Peer соединение:** установка прямого TCP-соединения между найденными устройствами[reference:8].
- **Альтернативный подход:** WebRTC с DataChannel для P2P-связи (обход NAT, работа через интернет)[reference:9][reference:10].
- **Обмен сообщениями:** текстовые сообщения, уведомления о статусе (онлайн/офлайн), история чата.
- **Кроссплатформенность:** .NET MAUI / Avalonia UI / Uno Platform для единого кода под Android и iOS.
- **Разрешения:** сетевые разрешения на мобильных платформах (Android, iOS)[reference:11].

- **Безопасность:** шифрование сообщений (опционально).

Практический пример: консольный или мобильный чат-клиент, который автоматически находит соседей в Wi-Fi сети и позволяет обмениваться сообщениями.

Ресурсы: Zeroconf (NuGet)[reference:12], примеры P2P-чатова на GitHub[reference:13][reference:14].

Веб-приложение "Менеджер задач" (Todo API) на ASP.NET Core

Цель: создать полноценный бэкенд для управления задачами с хранением в базе данных, авторизацией, логированием и интеграцией с внешним API. Это классический проект, который показывает, как работают вместе все ключевые Middle-технологии.

Подтемы:

- Проектирование моделей данных (Entity Framework Core, Code First, миграции)
- RESTful API на ASP.NET Core (контроллеры, маршрутизация, модель-биндинг, валидация)
- Dependency Injection (регистрация сервисов, работа с ILogger, IConfiguration)
- Работа с БД: PostgreSQL / SQLite / SQL Server через EF Core (запросы, LINQ, отношения)
- Аутентификация и авторизация (JWT, регистрация/вход пользователей)
- Кэширование (IMemoryCache, IDistributedCache + Redis)
- Логирование (Serilog / ILogger, structured logging, запись в файл или консоль)
- Фоновые задачи (IHostedService / BackgroundService) — например, очистка старых задач
- Интеграция с внешним API (например, получение курсов валют или погоды через HttpClient)
- Тестирование: Unit-тесты (xUnit + Moq), интеграционные тесты (TestContainers)
- Документирование API (Swagger / OpenAPI)

Практический пример: API для управления задачами с пользователями, категориями, приоритетами, сроками выполнения и уведомлениями.

Ресурсы: официальные tutorиалы Microsoft по ASP.NET Core, EF Core, JWT, xUnit, Serilog.

Практический проект: Универсальная панель управления сервисами (Service Manager)

Цель: Создать систему, которая позволяет управлять несколькими сервисами через единую веб-панель, без необходимости писать код для каждого нового сервиса. Всё, что нужно для интеграции — JSON-манифест, описывающий API сервиса.

Идея

Мы строим **Service Manager** — приложение, которое:

1. Хранит **реестр сервисов** (Service Registry) и их JSON-манифесты.
2. Предоставляет **веб-панель**, которая динамически генерирует UI на основе манифеста.
3. Позволяет **вызывать методы сервисов** через панель (прокси-запросы).
4. Умеет **регистрировать новые сервисы** без перезапуска (через API или файлы).



Компоненты системы

Компонент	Назначение
Service Registry	Хранит список сервисов и их манифестов. Умеет читать из папки <code>./Manifests</code> , принимать по HTTP, хранить в памяти или БД.
Manifest Parser	Парсит JSON-манифест, валидирует его схему, преобразует во внутренние объекты C#.
Dynamic UI Generator	На основе манифеста генерирует HTML-формы (или Blazor-компоненты) для каждого метода.
API Proxy	Принимает запросы от панели, формирует HTTP-запрос к целевому сервису, возвращает ответ.
Service Scanner	(Опционально) Фоновый сервис, который сканирует папку <code>./Manifests</code> и обновляет реестр при появлении новых файлов.



Формат манифеста (JSON)

```
{
  "serviceName": "UserService",
  "baseUrl": "https://user-api.example.com",
  "version": "1.0",
  "methods": [
    {
      "name": "GetUser",
      "httpMethod": "GET",
      "path": "/users/{id}",
      "parameters": [
        { "name": "id", "type": "integer", "required": true, "in": "path" }
      ],
      "responses": {
        "200": { "type": "object", "schema": { "$ref": "#/schemas/UserDto" } }
      }
    },
    {
      "name": "CreateUser",
      "httpMethod": "POST",
      "path": "/users",
      "parameters": [
        { "name": "body", "type": "object", "in": "body", "schema": { "$ref": "#/schemas/CreateUserDto" } }
      ]
    }
  ],
  "schemas": {
    "UserDto": {
      "type": "object",
      "properties": {
```

```

        "id": { "type": "integer" },
        "name": { "type": "string" },
        "email": { "type": "string" }
    },
    },
    "CreateUserDto": {
        "type": "object",
        "properties": {
            "name": { "type": "string" },
            "email": { "type": "string" }
        },
        "required": ["name", "email"]
    }
}
}
}

```

📁 Структура проекта

```

ServiceManager/
├── ServiceManager.sln
├── ServiceManager.Web/                                # Веб-панель (ASP.NET Core + Blazor/Razor)
│   ├── Program.cs
│   ├── appsettings.json
│   ├── Controllers/
│   │   └── ProxyController.cs                        # Прокси-запросы к сервисам
│   ├── Pages/                                        # (если Razor Pages)
│   │   ├── Index.cshtml                            # Список сервисов
│   │   └── Service.cshtml                          # Страница конкретного сервиса
│   (динамические формы)
│   │   └── Components/                              # (если Blazor)
│   ├── ServiceManager.Core/                          # Общие модели и логика
│   │   ├── Models/
│   │   │   ├── ServiceManifest.cs
│   │   │   ├── MethodDefinition.cs
│   │   │   └── ParameterDefinition.cs
│   │   ├── Services/
│   │   │   └── ServiceRegistry.cs                    # Реестр сервисов
│   │   ├── Parsers/
│   │   │   └── ManifestParser.cs                     # Парсинг JSON-манифеста
│   ├── ServiceManager.Scanner/                        # Фоновый сервис для сканирования
│   │   └── ManifestScannerService.cs                 # BackgroundService
│   Manifests/                                         # Папка для JSON-манифестов (сканируется)
│   │   ├── user-service.json
│   │   └── order-service.json

```

Текущая работа

Git и GitHub workflow, CI/CD

Управление версиями - краеугольный камень production разработки. Здесь не будет классических похвалб системам контроля версий, мол ты ничего не потеряешь, можешь откатиться и т.п., основная причина для использования систем контроля версий - кооперативная разработка. Они позволяют множеству разработчиков работать над единым проектом, а современные обёртки над ними (GitLab, Gitea, GitHub) позволяют следить за Issues, комментировать, обсуждать, коллаборироваться, распространять код, строить CI/CD и это куда более важные компоненты систем контроля версий, чем принято их преподносить начинающим разработчикам.

Git - мейнстрим контроля версий. По сути, он занимает такую долю рынка, что остальные системы, даже если существуют - не используются. Возможно, где-то в легаси можно ещё встретить старые не портированные системы, но они постепенно вымирают. Король один - Git.

Удалённые репозитории делятся на позорные (GitLab) и пацанские (GitHub, Gitea), отличие - императивность и декларативность. Чем более позорный твой удалённый репозиторий, тем больше он нравится бизнесу и более он декларативный. Для корп декларативность - плюс, для инженера - жопа.

В текущем конспекте будет рассматриваться связка **Gitea + GiteaActions** из-за полной совместимости с **Github actions**. **Gitlab CI** рассматриваться не будет, поскольку несовместим с **Github**.

Git - древовидная система контроля версий.

Репозиторий - директория в файловой системе, где находится **.git** поддиректория, содержащая файлы, описывающие репозиторий: коммиты, ветки, историю репозитория

Другие файлы - текущее состояние репозитория, т.е. конкретный набор файлов и директорий, которые относятся к активной ветке и активному коммиту.

Коммит - снимок репозитория, содержащий текущее на момент создания состояние файлов и директорий в репозитории

Ветка - история снимков, можно представить как стопку снимков состояний файлов и директорий. Несколько веток могут содержать разные истории, даже без общего "корня" (что плохо, так делать не надо, но техническая возможность есть). В нормальном репозитории все ветки коммитов сходятся в корень - момент инициализации репозитория. **Коммит** - узел этого дерева, откуда может начинаться другая ветка репозитория со своей историей изменений файлов.

В нормальном репозитории есть минимум 3 ветки: **master** - релизное состояние файлов, из которого собирается рабочая версия приложения для **production** среды (альт. назв: main, production, prod), **staging** - ветка, содержащая коммиты, в которых файлы отражают тестовую версию приложения, которая ещё не релиз, но уже тестируется (альт. назв.: **stage**, **staging**, **test**, **testing**). **develop** - среда разработки, сюда чаще всего вносятся изменения и здесь проводятся **unit** тесты. (альт. назв.: **dev**)

Дополнительно ветки могут быть что-то вроде **dev-feature-1**, относящиеся к реализации конкретной фичи назначенным разработчиком. Или **master-hotfix-15** - быстрые исправления критических ошибок в какой-либо ветке.

Структура дерева коммитов репозитория пример:

```

*   (main) Merge branch staging into main (релиз)
| \
| *   (staging) Подготовка к релизу (финальные правки)
| | \
| | *   (develop) Слияние feature/login в develop
| | | \
| | | *   (feature/login) Готовая фича входа (коммит E)
| | | *   Начальная реализация входа (коммит D)
| | | /
| | *   Изменение схемы БД (коммит C)
| | *   (develop создано из staging) Добавлены базовые модели (коммит B)
| | /
| *   (staging) Ветка staging создана от main (коммит B или C)
| /
*   (init) Инициализация репозитория (корневой коммит A)

```

Условные обозначения

- * — коммит (узел дерева).
- | / \ / — линии, показывающие родительские связи (кто от кого произошёл).
- (main), (staging), (develop) и т.д. — имена веток (указатели на конкретные коммиты).
- (HEAD) — текущая активная ветка (в примере не показана, чтобы не усложнять).
- Коммиты читаются снизу вверх — каждый новый коммит добавляется поверх предыдущего.

В реальных проектах может быть больше веток, но описанные (**main**, **staging**, **develop**, **feature/***, **hotfix/***) — это минимальный набор для контролируемой разработки.

Конфликты

Во время слияния веток может произойти конфликт, когда в ветке с которой происходит слияние есть изменения, которых нет в ветке из которой происходит слияние. В этом случае происходит ручное разрешение конфликта. Система контроля версий показывает 2 файла, файл в итоговой ветке и файл откуда происходит слияние, построчно сопоставляя их и подсвечивая изменения. В итоговом файле требуется привести его в разрешённое состояние, приняв решение, какие изменения оставить, а какие отклонить.

Вид того, как разрешается конфликт может отличаться, но обычно в итоговом файле требуется просто раскомментировать строки, содержащие слияние из исходной ветки или закомментировать то, что не требуется переносить из текущей ветки.

Команды Git

Настройки

```

# Установить имя пользователя (глобально для всех репозиториях)
git config --global user.name "Твоё Имя"

# Установить email (глобально для всех репозиториях)

```

```
git config --global user.email "твой.email@example.com"

# Для конкретного репозитория
# Перейти в папку репозитория
cd /путь/к/проекту

# Установить имя и email только для этого репозитория (без --global)
git config user.name "Рабочее Имя"
git config user.email "рабочий@company.com"

# Показать все текущие настройки
git config --list

# Показать только имя пользователя
git config user.name

# Показать только email
git config user.email

# Имя ветки по умолчанию
git config --global init.defaultBranch main
```

Минималка для первого коммита:

```
# Представиться Git (глобально)
git config --global user.name "Иван Иванов"
git config --global user.email "ivan@example.com"

# Установить основную ветку по умолчанию
git config --global init.defaultBranch main

# Проверить, что всё настроено
git config --list
```

Инициализация

1. Клон репозитория (обычно, когда репозиторий был создан на удалённой системе, вроде Github, GitLab или Gitea): `git clone https://url.to.remote/remote_user/remote_repo.git` - в директории, откуда была вызвана команда, будет создана директория `remote_repo`, куда будет добавлено содержимое репозитория и история веток.
2. Инициализация репозитория: `git init` - в текущей директории будет создана директория `.git` с initial коммитом. Репозиторий создаётся пустой.

Работа с текущими изменениями

```
# Посмотреть статус изменённых файлов
git status
```

```
# Добавить все изменения в индекс (подготовить к коммиту)
git add .
# или конкретный файл
git add Program.cs

# Создать коммит с сообщением
git commit -m "Добавлена валидация модели"

# Посмотреть историю коммитов
git log --oneline
```

Важно: сообщение коммита должно быть осмысленным. Писать тираду о том, в каком файле и что ты поменял не стоит, но указать "фикс валидации email" - вполне нормально

Коммиты должны быть как можно более атомарными, сделал одну микро-задачу - коммит. Коммит на каждую строчку - зря, а на микрозадачу - нормально.

Работа с удалённым репозиторием

```
# Добавить удалённый репозиторий
git remote add origin https://gitea.example.com/user/repo.git

# Отправить изменения в удалённый репозиторий
git push origin main

# Получить изменения с удалённого репозитория
git pull origin main
```

Работа с ветками

```
# Создать новую ветку и переключиться на неё
git checkout -b feature/new-validation

# Переключиться на другую ветку
git checkout main

# Показать все локальные ветки
git branch

# Показать все ветки (локальные и удалённые)
git branch -a

# Удалить локальную ветку
git branch -d feature/new-validation

# Переключиться на ветку, в которую будем сливать
git checkout main
```

```
# Слить ветку feature в текущую
git merge feature/new-validation
```

Конфликт возникает, когда одни и те же строки файла изменены в двух разных ветках. Git не может автоматически решить, какой вариант правильный, и помечает конфликтующие участки. Их нужно исправить вручную (выбрать один вариант или скомбинировать), затем добавить файл в индекс и сделать коммит слияния

Установка собственного удалённого репозитория

Потребуется система с **Linux**, либо **Windows** с **WSL** и пакет **Docker** для развёртывания системы контроля версий в контейнерном окружении.

Далее необходимо создать директорию для Gitea и создать в ней **docker-compose.yml** с примерно следующим содержанием:

```
services:
  gitea:
    image: gitea/gitea:latest
    container_name: gitea
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA__database__DB_TYPE=sqlite3
    restart: always
    volumes:
      - ./gitea:/data
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro
    ports:
      - "3000:3000"
      - "2222:22"
```

Далее в той же директории необходимо поднять Gitea:

```
docker compose up -d
```

эта команда запустит контейнер с Gitea в фоновом режиме и она будет доступна по адресу **http://localhost:3000**

Что здесь происходит:

- **volumes: ./gitea:/data** — данные сохраняются в папке **./gitea** на хосте, чтобы не потерять их при перезапуске контейнера.
- **ports: "3000:3000"** — веб-интерфейс.
- **ports: "2222:22"** — SSH-доступ к репозиториям (порт 22 на хосте может быть занят, поэтому используем 2222).

Далее, следует открыть станицу и первый, кто зарегистрируется будет администратором этой системы. После регистрации произойдёт попадание в панель администратора.

Важно конфигурация Gitea хранится в `app.ini` внутри контейнера. И для взаимодействия с конфигурационными файлами внутри контейнеров используется следующий алгоритм действий:

```
# Скопировать файл на хост
docker cp gitea:/data/gitea/conf/app.ini ./app.ini

# Отредактировать (например, включить Actions, о чём в части 2)
nano app.ini # Я рекомендую попробовать micro, он чуть лучше nano по
пользовательскому опыту, но может иметь не полную поддержку.

# Скопировать обратно
docker cp ./app.ini gitea:/data/gitea/conf/app.ini

# Перезапустить Gitea
docker restart gitea
```

push-to-create - удобная фишка, которая позволяет авторизованному пользователю создавать репозиторий путём пуша, а не через веб-панель.

В `app.ini` следует добавить в секцию `[repository]` следующее:

```
[repository]
ENABLE_PUSH_CREATE_USER = true
ENABLE_PUSH_CREATE_ORG = true
```

Если нужно, чтобы новосозданные репозитории были приватны:

```
DEFAULT_PUSH_CREATE_PRIVATE = true
```

после этого: `docker restart gitea` чтобы перезапустить сервис и применить конфигурацию.

Рабочий пайплайн. Git Workflow. Алгоритм работы с репозиторием.

Допустим, в команде начинается работа над проектом `my-cool-app`;

Этап 1.

Создание проекта. Для начала Gitea настраивается на `push-to-create`.

```
# Создаём директорию проекта
mkdir my_new_app
# Инициализируем репозиторий (ВАЖНО: репозитории инициализируются в пустых
директориях)
```



```
git init
git add .
git commit -m "initial" # -m параметр позволяет задать описание коммита без
открытия текстового редактора

# Добавляем удалённый репозиторий, после неё репозиторий ещё не создан, мы просто
указали git на локальном компьютере куда необходимо пушить будет.
git remote add origin http://gitea.localserver.domain/username/my_new_app.git

# Пушим в мастер на origin
git push -u origin master

# Теперь только создаём сам проект
dotnet new sln -n "MyNewApp"
dotnet new console -n "MyNewConsoleApp"
dotnet sln MyNewApp add ./MyNewConsoleApp/MyNewConsoleApp.csproj

# Структура проекта создана, теперь можно отправить его в репозиторий
git add .
git commit -m "Added dotnet solution and linked console project"
git push

# Теперь, зайдя в панель Gitea можно будет увидеть, что появился репозиторий с 2
коммитами: инициализационный и коммит с добавлением решения и проекта.
```

Этап 2.

Далее требуется создать **минимальную** структуру репозитория, которая соответствует рекомендациям по работе с версиями.

Важно Это не догмат, а очень важная и значимая рекомендация для работы над длительными серьёзными проектами. Для личных пет проектов или баловства, тестов или чего-то подобного, что не подразумевает длительной разработки, не подразумевает коллаборации, не подразумевает расширения и развития или является proof-of-concept применение этой структуры ведёт к managedgement-overhead и делать так не надо.

```
# Создаём stage ветку, для тестирование сборок
git checkout -b staging
git add .
git commit -m "add staging"
git push origin staging
# Создаём develop ветку, для тестирование сборок
git checkout -b developing
git add .
git commit -m "add developing"
git push origin developing
```

Далее работа над приложением продолжается с ветвления от ветки **develop**.

Чтобы назначить ветку по-умолчанию над которой сейчас осуществляется работа, чтобы постоянно не указывать, куда пушить, используется опция `--set-upstream`

`git push --set-upstream origin developing` - коммиты по умолчанию командой `git push` без указания ветки будут пушиться в `develop`

Этап 3.

Теперь происходит работа над некоторой фичей, для чего создаётся новая ветка.

Важно Для личных проектов это не обязательно, но рекомендовано, если вашим личным проектом пользуются люди. Даже если ветка всего одна - `master` (`main`), ветку для разработки очередной версии приложения имеет смысл делать отдельную, чтобы люди, которые будут пользоваться и собирать ВАШ софт не столкнулись с проблемами, которые связаны с тем, что ваша сборка не очень стабильна или вовсе не собирается. В `master` ветке публичного репозитория, которым будут пользоваться другие люди должна быть та версия файлов, которая успешно собирается и не создаёт проблем с основной функциональности.

```
# отделяем ветку от develop для новой фичи

git checkout -b my_feature_1
git add .
git commit -m "add my_feature_1"
git push --set-upstream origin my_feature_1

# Далее после осуществления работы и коммитов в ветку, мы принимаем решение о
# слиянии с develop веткой

git checkout develop
git merge my_feature_1

# В соло разработке или разработке, когда никто не вносил изменения в ветку
# develop всё произойдёт без проблем.
# Однако, если кто-то в ветке develop произведёт изменение тех же файлов, что
# меняли вы в ходе своей работы, то возникнет конфликт
```

Этап 4.

Решение конфликтов.

Git сообщит о конфликте при слиянии веток, в которых есть изменения в одних и тех же файлах. Открой файл в редакторе, найди участки с маркерами:

```
<<<<<< HEAD
// код из main
=====
```

```
// код из feature
>>>>> feature/add-logging
```

Далее требуется снять маркеры с изменений, которые требуется зафиксировать в текущем коммите и сохранить файл, после повторить добавление в индекс и сделать коммит:

```
git add .
git commit -m "merge"
git push origin main # для примера main
```

Этап 5.

Периодически приходится пересматривать историю веток и искать проблемный коммит, для чего есть следующие команды.

```
# Показать историю в одну строку
git log --oneline
# В виде дерева
git log --graph
# --decorate добавляет читабельности
git log --graph --decorate
# Базовый древовидный вывод с указанием веток и сокращённых хешей
git log --graph --oneline --decorate

# Более детальный с датами и авторами
git log --graph --pretty=format:'%Cred%h%Creset - %s %Cgreen(%cr) %C(bold blue)
<%an>%Creset' --abbrev-commit

# Показать все ветки в дереве
git log --graph --oneline --decorate --all

# Ограничить количество коммитов (например, последние 20)
git log --graph --oneline --decorate -20

# Найти коммит, который сломал тесты
git bisect start
git bisect bad HEAD
git bisect good a1b2c3d
# ... и так далее
```

Типовая структура репозитория

```
my-app/
├── .gitignore          # Что не попадает в репозиторий (bin, obj, packages)
├── .gitea/             # Папка для Gitea Actions (часть 2)
│   ├── workflows/
│   └── ci-cd.yml
```

```
|— src/                                # Исходный код
|   └─ MyApp/
|       ├── MyApp.csproj
|       └─ Program.cs
|— tests/                             # Тесты
|   └─ MyApp.Tests/
|— Dockerfile                         # Инструкция для сборки образа (часть 3)
|— docker-compose.yml                 # Для локального запуска (часть 3)
|— README.md
```

.gitignore - специальный файл, который указывает, какие расширения файлов НЕ будут попадать в индекс (отладочная информация, секреты и прочее, что может компромитировать разработку)

Ниже **.gitignore** для .Net в минимальном варианте:

```
bin/
obj/
packages/
*.user
*.suo
*.pdb
*.log
```